Lab 4 – Vibe Coding: Word Scramble (62_scramble) – LLM Chat History

Student: Rehan S. Bawakhan · SRN: PES2UG25CS820 · Section J
LLM used: Claude (Anthropic), model claude-opus-5-5 · Date: 15 October 2026
Repo: https://github.com/SETAPESU26/62_scramble
Note: screenshots are described in brackets; Claude's tool actions (cloning, testing) are summarised as steps.

REHAN

[Attached: Lab_4_VibeCoding_Student_handout.pdf]

https://github.com/SETAPESU26/62_scramble

CLAUDE

On it, mate. Cloning the repo to see the code and README.

> **Steps taken:** cloned the repo; read `README.md`, `main.py`, `game/game_engine.py`, `game/text_box.py`. Found the bug in `submit_guess()`: `is_correct = (guess == self.scrambled_word)` compares the guess against the **scrambled** text instead of the secret word. Rewrote `game_engine.py` (bug fix + hints + timer + tiles), fixed the window caption in `main.py` ("Anvil Dodge" → "Word Scramble Arena"), then ran headless tests.

Test results:

```
scrambled rejected: True
correct accepted: True
hint locks correct: True worth 7
max hints 5 of 6
swap ok: True
use tiles: True
timeout: TIME'S UP! The word was 'PUZZLE'. True
```

All four tasks are done and tested headless, mate. `text_box.py` is unchanged.

Order matters for the videos: record the "before" clip on the original code first, then swap in `game_engine.py` and `main.py`.

What changed:

1. **Bug fix:** the check was `guess == self.scrambled_word`, so typing the scrambled letters won. It's now `guess == self.secret_word`.
2. **Hints:** use the HINT button or Tab. Each hint moves the correct letter into its slot and locks it green. A word starts at 10 pts, and each hint removes one letter's share (`10 × (n − hints) / n`). You always have to solve the last letter yourself.
3. **Timer:** there's a 30s bar that turns green, then amber, then red. When it runs out, the game shows "TIME'S UP! The word was X" and moves to a new word.
4. **Tiles:** click two tiles to swap them. SHUFFLE TILES reshuffles the unlocked tiles. USE TILE ORDER copies your arrangement into the input box so you can submit it.

5. **Smaller fixes:** the window title said "Anvil Dodge", so I renamed it. Clicking a button no longer takes focus away from the text box.

REHAN

so where did u edit it

CLAUDE

I edited a copy of the repo in my own cloud workspace, not on your laptop, so nothing on your machine has changed yet. The two files I sent you (`game_engine.py` and `main.py`) are the edited versions.

Where the edits are:

- **main.py:** one line, the `set_caption("Word Scramble Arena")` call (it used to say "Anvil Dodge").
- **game_engine.py:** - **Bug fix:** in `submit_guess()`, `is_correct = (guess == self.secret_word)` - **Hints:** the new `use_hint()` and `round_value()` methods - **Timer:** `update()` and the timer bar section in `render()` - **Tiles:** the new `click_tile()`, `shuffle_tiles()`, `use_tiles()` and `_layout_tiles()` methods, plus the tile drawing loop in `render()`

REHAN

ok so i need to clone it and run the game and tale clips >??

CLAUDE

Yep, mate. Here's the order on Windows:

1. Clone and set up: `git clone https://github.com/SETAPESU26/62_scramble.git`, `cd 62_scramble`, `pip install pygame`
2. Record the **before** clip on the original code (`python main.py`, Win + Alt + R): type the correct word → rejected; type the scrambled letters → accepted.
3. Swap in the two new files.
4. Record the **after** clip: timer draining, HINT locking a green tile, swapping tiles, USE TILE ORDER → SUBMIT → "CORRECT".
5. Clips save to `Videos\Captures`; rename to `before.mp4` / `after.mp4`.

REHAN

[Screenshot: `pip install pygame` fails — "Failed to build 'pygame' when getting requirements to build wheel", ModuleNotFoundError: No module named 'setuptools._distutils.msvccompiler', Python 3.14]

CLAUDE

Your Python is 3.14, and regular `pygame` has no prebuilt version for it yet. That's why pip tries to build it from source and fails.

Install **pygame-ce** instead. It's the maintained fork, it supports 3.14, and you still write `import pygame`, so no code changes are needed:

```
pip install pygame-ce
python main.py
```

If that also fails, use Python 3.12: `py -3.12 -m pip install pygame` then `py -3.12 main.py`.

REHAN

[Screenshot: pygame-ce 2.5.8 installed successfully; `python main.py` says "Python was not found; run without arguments to install from the Microsoft Store"]

CLAUDE

pygame-ce installed fine. The `python` command is hitting a Microsoft Store shortcut instead of your real Python. Use the launcher instead:

```
py main.py
```

To make `python` work too: Settings → Apps → Advanced app settings → App execution aliases, turn off "python.exe" and "python3.exe".

REHAN

ha done after this

CLAUDE

Game's running, so now do the rest in this order:

1. Record the before clip on the original code.
2. Swap in the new files.
3. Record the after clip.
4. Rename clips to `before.mp4` and `after.mp4`.
5. Export this chat as a PDF.
6. Push everything to your lab repo under `Lab-4/`.

REHAN

u give me code i will copy paste

CLAUDE

Here's the full code, mate. Replace everything in each file.

`game\game_engine.py`

```
import random
import pygame
from game.text_box import TextBox

ROUND_TIME = 30.0          # seconds per round
WORD_POINTS = 10           # points for a word solved with no hints
TILE_SIZE = 54
TILE_GAP = 8


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.words = ["PYTHON", "PYGAME", "PLANET", "ROCKET", "GALAXY", "STREAM",
"PUZZLE", "ALGORITHM"]
        self.secret_word = ""
        self.scrambled_word = ""

        self.score = 0
        self.feedback_msg = "Unscramble the letters above!"
        self.feedback_color = (210, 215, 225)

        # Hint state
        self.hints_used = 0
        self.locked = set()          # tile indices fixed in place by hints

        # Tile state
        self.tiles = []              # current letter order shown as tiles
        self.tile_rects = []
        self.selected_tile = None

        # Timer state
        self.time_left = ROUND_TIME
        self.last_tick = pygame.time.get_ticks()

        # Layout
        self.input_box = TextBox(width // 2 - 200, 235, 190, 46)
        self.submit_btn = pygame.Rect(width // 2 + 0, 235, 95, 46)
        self.hint_btn = pygame.Rect(width // 2 + 105, 235, 95, 46)
        self.use_tiles_btn = pygame.Rect(width // 2 - 200, 295, 190, 40)
        self.shuffle_btn = pygame.Rect(width // 2 + 0, 295, 200, 40)

        self.font_title = pygame.font.SysFont(None, 40)
        self.font_tile = pygame.font.SysFont(None, 44)
        self.font_msg = pygame.font.SysFont(None, 26)
        self.font_btn = pygame.font.SysFont(None, 24)
        self.font_small = pygame.font.SysFont(None, 20)

        self.next_round()

    # ------------------------------------------------------------------ setup
    def scramble_string(self, word):
        letters = list(word)
        while True:
            random.shuffle(letters)
            shuffled = "".join(letters)
            if shuffled != word or len(word) <= 1:
                return shuffled

    def next_round(self):
        self.secret_word = random.choice(self.words)
        self.scrambled_word = self.scramble_string(self.secret_word)
        self.tiles = list(self.scrambled_word)
        self.selected_tile = None
        self.locked = set()
        self.hints_used = 0
        self.time_left = ROUND_TIME
```

```
        self.last_tick = pygame.time.get_ticks()
        self.input_box.clear()
        self._layout_tiles()

    def _layout_tiles(self):
        n = len(self.tiles)
        total = n * TILE_SIZE + (n - 1) * TILE_GAP
        start_x = self.width // 2 - total // 2
        self.tile_rects = [
            pygame.Rect(start_x + i * (TILE_SIZE + TILE_GAP), 145, TILE_SIZE, TILE_SIZE)
            for i in range(n)
        ]

    # ------------------------------------------------------------------ scoring
    def round_value(self):
        """Proportional penalty: each hint removes one letter's share of the points."""
        n = len(self.secret_word)
        return max(1, round(WORD_POINTS * (n - self.hints_used) / n))

    def submit_guess(self):
        guess = self.input_box.text.strip().upper()
        if not guess:
            self.feedback_msg = "Type a word before submitting!"
            self.feedback_color = (240, 170, 50)
            return

        # Task 1 fix: compare against the secret word, not the scrambled display
        is_correct = (guess == self.secret_word)

        if is_correct:
            gained = self.round_value()
            self.score += gained
            self.feedback_msg = f"CORRECT! '{self.secret_word}' is right. +{gained}"
            self.feedback_color = (80, 230, 110)
            self.next_round()
        else:
            self.feedback_msg = "WRONG GUESS! Try again."
            self.feedback_color = (240, 80, 80)
            self.input_box.clear()

    # ------------------------------------------------------------------ hints
    def use_hint(self):
        n = len(self.secret_word)
        unrevealed = [i for i in range(n) if i not in self.locked]
        if len(unrevealed) <= 1:
            self.feedback_msg = "No more hints for this word!"
            self.feedback_color = (240, 170, 50)
            return

        pos = random.choice(unrevealed)
        needed = self.secret_word[pos]

        # Move a matching, unlocked tile into the correct position, then lock it
        if self.tiles[pos] != needed:
            for j in range(n):
                if j not in self.locked and j != pos and self.tiles[j] == needed:
                    self.tiles[pos], self.tiles[j] = self.tiles[j], self.tiles[pos]
                    break
        self.locked.add(pos)
        self.selected_tile = None
        self.hints_used += 1

        self.feedback_msg = (f"Hint: letter {pos + 1} is '{needed}'. "
                             f"Word now worth {self.round_value()} pts")
        self.feedback_color = (255, 220, 80)

    # ------------------------------------------------------------------ tiles
    def click_tile(self, idx):
        if idx in self.locked:
            return
        if self.selected_tile is None:
            self.selected_tile = idx
```

```
        elif self.selected_tile == idx:
            self.selected_tile = None
        else:
            a, b = self.selected_tile, idx
            self.tiles[a], self.tiles[b] = self.tiles[b], self.tiles[a]
            self.selected_tile = None

    def shuffle_tiles(self):
        free = [i for i in range(len(self.tiles)) if i not in self.locked]
        letters = [self.tiles[i] for i in free]
        random.shuffle(letters)
        for i, ch in zip(free, letters):
            self.tiles[i] = ch
        self.selected_tile = None

    def use_tiles(self):
        self.input_box.text = "".join(self.tiles)

    # ------------------------------------------------------------------ events
    def handle_event(self, event):
        self.input_box.handle_event(event)

        if event.type == pygame.KEYDOWN:
            if event.key in (pygame.K_RETURN, pygame.K_KP_ENTER):
                self.submit_guess()
            elif event.key == pygame.K_TAB:
                self.use_hint()
        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            clicked_ui = True
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()
            elif self.hint_btn.collidepoint(event.pos):
                self.use_hint()
            elif self.use_tiles_btn.collidepoint(event.pos):
                self.use_tiles()
            elif self.shuffle_btn.collidepoint(event.pos):
                self.shuffle_tiles()
            else:
                clicked_ui = False
                for i, rect in enumerate(self.tile_rects):
                    if rect.collidepoint(event.pos):
                        self.click_tile(i)
                        clicked_ui = True
                        break
            # keep typing focus after clicking any game control
            if clicked_ui:
                self.input_box.active = True

    # ------------------------------------------------------------------ update
    def update(self):
        now = pygame.time.get_ticks()
        self.time_left -= (now - self.last_tick) / 1000.0
        self.last_tick = now

        if self.time_left <= 0:
            word = self.secret_word
            self.next_round()
            self.feedback_msg = f"TIME'S UP! The word was '{word}'."
            self.feedback_color = (240, 80, 80)

    # ------------------------------------------------------------------ render
    def _button(self, screen, rect, label, color):
        pygame.draw.rect(screen, color, rect, border_radius=6)
        pygame.draw.rect(screen, (220, 220, 220), rect, width=2, border_radius=6)
        txt = self.font_btn.render(label, True, (255, 255, 255))
        screen.blit(txt, (rect.centerx - txt.get_width() // 2, rect.centery -
txt.get_height() // 2))

    def render(self, screen):
        screen.fill((26, 30, 38))

        title_surf = self.font_title.render("Word Scramble Arena", True, (245, 245,
```

```
245))
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 25))

        score_surf = self.font_msg.render(
            f"Score: {self.score}    Hints: {self.hints_used}    Worth:
{self.round_value()} pts",
            True, (255, 220, 80))
        screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 70))

        # Timer bar
        bar_w, bar_h = 460, 14
        bar_x, bar_y = self.width // 2 - bar_w // 2, 105
        frac = max(0.0, self.time_left / ROUND_TIME)
        if frac > 0.5:
            bar_color = (80, 200, 120)
        elif frac > 0.2:
            bar_color = (240, 190, 60)
        else:
            bar_color = (235, 75, 75)
        pygame.draw.rect(screen, (55, 60, 72), (bar_x, bar_y, bar_w, bar_h),
border_radius=7)
        pygame.draw.rect(screen, bar_color, (bar_x, bar_y, int(bar_w * frac), bar_h),
border_radius=7)
        t_surf = self.font_small.render(f"{max(0, int(self.time_left + 0.999))}s", True,
(230, 230, 230))
        screen.blit(t_surf, (bar_x + bar_w + 10, bar_y - 1))

        # Letter tiles
        for i, rect in enumerate(self.tile_rects):
            if i in self.locked:
                fill, border = (60, 150, 90), (140, 240, 170)
            elif i == self.selected_tile:
                fill, border = (240, 190, 60), (255, 240, 180)
            else:
                fill, border = (45, 85, 140), (100, 200, 255)
            draw_rect = rect.move(0, -6) if i == self.selected_tile else rect
            pygame.draw.rect(screen, fill, draw_rect, border_radius=8)
            pygame.draw.rect(screen, border, draw_rect, width=2, border_radius=8)
            letter = self.font_tile.render(self.tiles[i], True, (255, 255, 255))
            screen.blit(letter, (draw_rect.centerx - letter.get_width() // 2,
                                 draw_rect.centery - letter.get_height() // 2))

        self.input_box.render(screen)

        self._button(screen, self.submit_btn, "SUBMIT", (50, 150, 85))
        self._button(screen, self.hint_btn, "HINT", (190, 130, 40))
        self._button(screen, self.use_tiles_btn, "USE TILE ORDER", (70, 90, 160))
        self._button(screen, self.shuffle_btn, "SHUFFLE TILES", (110, 80, 150))

        feedback_surf = self.font_msg.render(self.feedback_msg, True,
self.feedback_color)
        screen.blit(feedback_surf, (self.width // 2 - feedback_surf.get_width() // 2,
365))

        help_lines = [
            "Click two tiles to swap them. Green tiles are locked hints.",
            "Enter = submit   |   Tab = hint   |   Each hint costs one letter's share of
the points",
        ]
        for k, line in enumerate(help_lines):
            s = self.font_small.render(line, True, (140, 145, 160))
            screen.blit(s, (self.width // 2 - s.get_width() // 2, 440 + k * 22))
```

main.py ```python import pygame from game.game_engine import GameEngine

WIDTH, HEIGHT = 700, 500 FPS = 60

def main(): pygame.init() screen = pygame.display.set_mode((WIDTH, HEIGHT)) pygame.display.set_caption("Word Scramble Arena") clock = pygame.time.Clock()

```
engine = GameEngine(WIDTH, HEIGHT)

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        engine.handle_event(event)

    engine.update()
    engine.render(screen)

    pygame.display.flip()
    clock.tick(FPS)

pygame.quit()
```

if **name** == "**main**": main()```

Record the before clip first, then paste these in, then run `py main.py`.

REHAN

ok i pasted and whats next

CLAUDE

If you didn't record the before clip, get the original code back temporarily with `git stash`, run `py main.py`, record, then `git stash pop`.

Then record the after clip with `py main.py`: timer draining, HINT turning a tile green, swapping two tiles, typing the right word → CORRECT.

REHAN

after that mate

CLAUDE

1. Export this chat as `chat_history.pdf`.
2. Create/clone your lab repo (`SE-Lab`), make `Lab-4`, copy in `game\`, `main.py`, `README.md`.
3. Drag in `before.mp4`, `after.mp4`, `chat_history.pdf`.
4. `git add Lab-4`, `git commit -m "Lab 4: fix scramble guess bug, add hints, timer, letter tiles"`, `git push`.

REHAN

[Screenshot: created Lab-4, copied files, then ran `git clone` again inside SE-Lab] im unable to see the folder

CLAUDE

The folder is there, mate. You haven't pushed yet, so it won't show on GitHub. You also ran `git clone` a second time, which made an extra `SE-Lab` folder inside your repo. Clean up:

```
Remove-Item -Recurse -Force SE-Lab
Remove-Item -Recurse -Force Lab-4\game\__pycache__
explorer Lab-4
```

Drag the videos and PDF in, then `git add Lab-4`, commit, `git push`.

REHAN

[Screenshot: README "Tasks to Complete" — Tasks 1 to 4] is it all done ??

CLAUDE

Yes, all four are done in the code you pasted:

Task	Done	How it works in the game
1. Guess bug	✅	Checks against `secret_word`, so the correct word is accepted and the scrambled letters are rejected
2. Hint system	✅	HINT button or Tab locks one correct letter in place (green). Each hint takes one letter's share off the 10 points
3. Countdown timer	✅	A 30s bar goes green → amber → red. When it runs out, the word is revealed and a new round starts
4. Letter tiles	✅	Click two tiles to swap them. SHUFFLE TILES and USE TILE ORDER let you submit your arrangement

REHAN

ok so i want chat history so make it

CLAUDE

Making the chat history PDF now — this document.